

Technical Report: Image Classification System using EfficientNet-B0 and FastAPI

1. Dataset Used

The selected dataset is CIFAR-10, a publicly available image classification dataset containing 60,000 RGB images across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The dataset satisfies the assignment requirement because it contains more than 5 classes. The data is split into training, validation, and testing sets. The original training set is divided into 80% training and 20% validation, while the official test set is used for final evaluation.

2. Model Selected

EfficientNet-B0 is selected as the classification model. It is a lightweight convolutional neural network that provides a good balance between accuracy and computational cost. ImageNet pretrained weights are used for transfer learning, and the final classification layer is replaced with a new linear layer for 10 CIFAR-10 classes.

3. Training Approach

Images are resized to 224 x 224 pixels to match the EfficientNet input size. The preprocessing pipeline includes tensor conversion and ImageNet normalization. Data augmentation is applied using random horizontal flip and random rotation to improve generalization. CrossEntropyLoss is used as the loss function and Adam optimizer is used for training. The best model is saved based on validation accuracy.

4. Evaluation

The model is evaluated on the test set using accuracy, precision, recall, and F1-score. After running the provided Colab notebook or train.py file, the exact test accuracy and classification report are printed in the output. The trained model is saved as models/efficientnet_b0_cifar10.pth.

5. API Development

A FastAPI application is implemented with a POST /predict endpoint. The API accepts an uploaded image, preprocesses it, performs model inference, and returns a JSON response containing the predicted class name and confidence score. A GET / endpoint is also provided to check whether the API is running.

6. Challenges Faced

The main challenges include adapting CIFAR-10 images to the input size expected by EfficientNet, avoiding overfitting on a relatively small image dataset, and ensuring that the same preprocessing is used during both training and inference. Another challenge is packaging the trained model correctly so that FastAPI can load it reliably.

7. Improvements with More Time

With additional time, performance can be improved by training for more epochs, using learning rate scheduling, experimenting with ConvNeXt or Vision Transformer models, applying stronger data augmentation, using model quantization for faster inference, and deploying the API on a cloud platform such as Render, Railway, or AWS.

8. Sample API Response

{ "prediction": "ship", "confidence": 0.94 }

Component	Choice
Dataset	CIFAR-10
Classes	10
Model	EfficientNet-B0
Framework	PyTorch
API	FastAPI
Output	Prediction and confidence score